

C-PROGRAMMING

TPE-004

TPE-004 The Basics of C

Learn the fundamental building blocks of programming: data types, variables, constants, operators, and basic input/output using *printf* and *scanf*.

Contact

svandamme@omnesintervenant.com

Table of content

TPE-004-1 | Introduction to Data and Variables

**Understand what data is in programming and how C stores it using variables.
Learn how to declare, initialize, and use variables in simple programs.**

TPE-004-2 | Constants and Literals

**Learn how to represent fixed values in programs and prevent changes to
important data using constants.**

TPE-004-3 | Operators and Expressions

**Master the symbols that let you perform calculations and comparisons.
Understand how C evaluates expressions.**

TPE-004-4 | Input and Output (I/O) Basics

**Use the standard input/output functions to interact with users. Learn how to
display results and read user input.**

Variables

>

What's a variable

TPE-004-1 | Introduction to Data and Variables

Understand what data is in programming and how C stores it using variables.

Learn how to declare, initialize, and use variables in simple programs.

Variables



Overview

Type category	Type (C-syntax)	Format specifier ¹
Integer	short	%hi
	unsigned short	%hi
	int	%i
	unsigned int	%u
	long	%li
	unsigned long	%lu
	long long	%lli
	unsigned long long	%llu
Character (processed as integer)	char	%c
	signed char	%c
	unsigned char	%c
Real (decimal)	float	%f
	double	%lf
	long double	%lf
Boolean	bool	%d

How to set a variable?

```
// INITIALIZE:  
int x;  
  
// USE!  
x = 2;  
x = x + 4;
```

```
// Or both directly. >:)  
float pi = 3.13159;
```

Variables

>

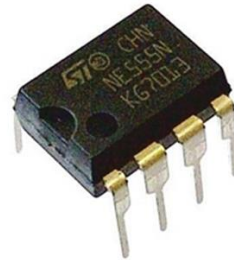
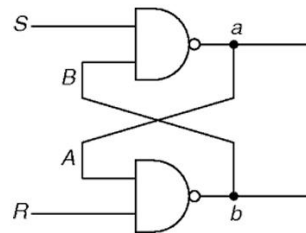
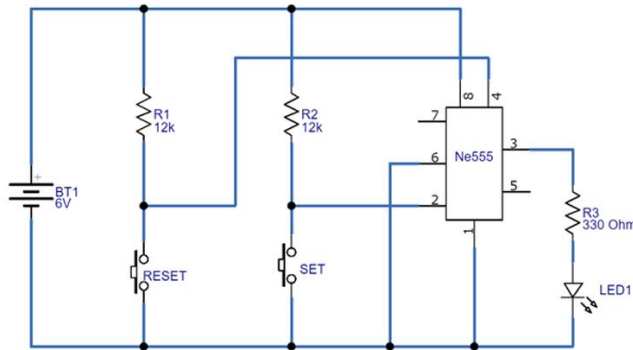
A “bit” of memory...

A **binary** variable can have only **two values**:

1	0
---	---

Now, using multiple binary slot in the memory of the machine, we can code on multiple bits!

One Bit Memory



Analogy with **4 decimal digits**:

$$\begin{array}{r}
 (X)_{10} = \begin{array}{|c|c|c|c|} \hline 9 & 5 & 0 & 3 \\ \hline \end{array} \\
 \text{(4 digits)} \quad \begin{array}{cccc} 10^4 & 10^2 & 10^1 & 10^0 \end{array} \quad \text{Multiplied by...} \\
 \hline
 \begin{array}{cccc} 9000 & 500 & 00 & 3 \end{array} \quad \text{Result per tile} \\
 \hline
 \text{Total sum (in base-10): } \mathbf{9503}
 \end{array}$$

Coding in **4 bits**:

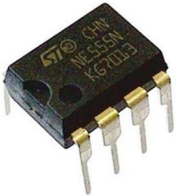
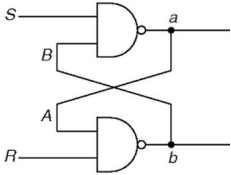
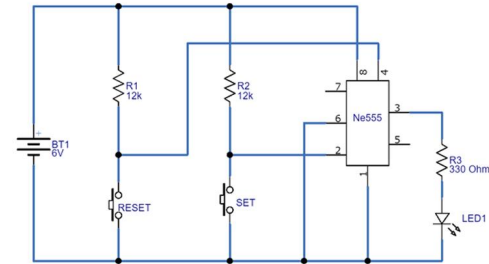
$$\begin{array}{r}
 (X)_2 = \begin{array}{|c|c|c|c|} \hline 1 & 1 & 1 & 0 \\ \hline \end{array} \\
 \text{(4 bits)} \quad \begin{array}{cccc} 2^3 & 2^2 & 2^1 & 2^0 \end{array} \quad \text{Multiplied by...} \\
 \hline
 \begin{array}{cccc} 8 & 4 & 2 & 0 \end{array} \quad \text{Result per tile} \\
 \hline
 \text{Total sum (in base-10): } \mathbf{14}
 \end{array}$$

Variables

>

Boolean algebra (1/2)

One Bit Memory

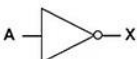


Logic Gate	Symbol	OCR Notation	AQA Notation	Edexcel Notation
AND		$A \wedge B$	$A \times B$	$A . B$
OR		$A \vee B$	$A + B$	$A + B$
NOT		$\neg A$	$\overline{A}$	$\overline{A}$
NAND		$\neg (A \wedge B)$	$\overline{A \times B}$	$\overline{A . B}$
NOR		$\neg (A \vee B)$	$\overline{A + B}$	$\overline{A + B}$
XOR		$A \underline{\vee} B$	$A \oplus B$	$A \oplus B$
XNOR		$\neg (A \underline{\vee} B)$	$\overline{A \oplus B}$	$\overline{A \oplus B}$

Variables

>

Boolean algebra (2/2)

Logic Gate	Symbol	OCR Notation	AQA Notation	Edexcel Notation
AND		$A \wedge B$	$A \times B$	$A . B$
OR		$A \vee B$	$A + B$	$A + B$
NOT		$\neg A$	$\overline{A}$	$\overline{A}$
NAND		$\neg (A \wedge B)$	$\overline{A \times B}$	$\overline{A . B}$
NOR		$\neg (A \vee B)$	$\overline{A + B}$	$\overline{A + B}$
XOR		$A \underline{\vee} B$	$A \oplus B$	$A \oplus B$
XNOR		$\neg (A \underline{\vee} B)$	$\overline{A \oplus B}$	$\overline{A \oplus B}$

Name	AND form	OR form
Identity law	$1A = A$	$0 + A = A$
Null law	$0A = 0$	$1 + A = 1$
Idempotent law	$AA = A$	$A + A = A$
Inverse law	$A\overline{A} = 0$	$A + \overline{A} = 1$
Commutative law	$AB = BA$	$A + B = B + A$
Associative law	$(AB)C = A(BC)$	$(A + B) + C = A + (B + C)$
Distributive law	$A + BC = (A + B)(A + C)$	$A(B + C) = AB + AC$
Absorption law	$A(A + B) = A$	$A + AB = A$
De Morgan's law	$\overline{AB} = \overline{A} + \overline{B}$	$\overline{A + B} = \overline{A}\overline{B}$

A **binary** variable can have only **two values**:

0	1
---	---

An **octet** is a set of 8 bits put together.

A **kilo-octet** (ko) is NOT the same as a **kilo-bit** (kb)

A **decimal** variable can have up to **10 values** per tile:

0	1	2	3	...	8	9
---	---	---	---	-----	---	---

An **hexadecimal** variable can have up to **16 values** per tile:

0	1	2	3	...	8	9	A	B	C	D	E	F
							10	11	12	13	14	15

Binary Addition and Subtraction

Add or Subtract the given Binary numbers.

$$\begin{array}{r} 1) \quad 11011010 \\ - 11010111 \\ \hline \end{array}$$

$$\begin{array}{r} 2) \quad 11010111 \\ - 01101100 \\ \hline \end{array}$$

$$\begin{array}{r} 3) \quad 10001100 \\ - 01101100 \\ \hline \end{array}$$

$$\begin{array}{r} 4) \quad 10001100 \\ + 10010010 \\ \hline \end{array}$$

$$\begin{array}{r} 5) \quad 10010010 \\ + 10010111 \\ \hline \end{array}$$

$$\begin{array}{r} 6) \quad 10010111 \\ + 11000101 \\ \hline \end{array}$$

$$\begin{array}{r} 7) \quad 11100011 \\ - 11000101 \\ \hline \end{array}$$

$$\begin{array}{r} 8) \quad 11100011 \\ - 01111000 \\ \hline \end{array}$$

$$\begin{array}{r} 9) \quad 10010000 \\ - 01111000 \\ \hline \end{array}$$

$$\begin{array}{r} 10) \quad 10010000 \\ + 11000100 \\ \hline \end{array}$$

$$\begin{array}{r} 11) \quad 11000100 \\ + 01011001 \\ \hline \end{array}$$

$$\begin{array}{r} 12) \quad 01011001 \\ + 01100100 \\ \hline \end{array}$$

$$\begin{array}{r} 13) \quad 01100100 \\ + 11010001 \\ \hline \end{array}$$

$$\begin{array}{r} 14) \quad 11011111 \\ - 11010001 \\ \hline \end{array}$$

$$\begin{array}{r} 15) \quad 11101000 \\ - 11011111 \\ \hline \end{array}$$

$$\begin{array}{r} 16) \quad 11101000 \\ + 01111100 \\ \hline \end{array}$$

Variables

>

Binary calculations

Converting Decimal and Binary Numbers

Convert the given Decimal number to its Binary equivalent.

- 1) $145_{(10)} = \underline{\hspace{2cm}}_{(2)}$ 2) $124_{(10)} = \underline{\hspace{2cm}}_{(2)}$
- 3) $242_{(10)} = \underline{\hspace{2cm}}_{(2)}$ 4) $116_{(10)} = \underline{\hspace{2cm}}_{(2)}$
- 5) $182_{(10)} = \underline{\hspace{2cm}}_{(2)}$ 6) $70_{(10)} = \underline{\hspace{2cm}}_{(2)}$
- 7) $238_{(10)} = \underline{\hspace{2cm}}_{(2)}$ 8) $164_{(10)} = \underline{\hspace{2cm}}_{(2)}$

Convert the given Binary to its Decimal equivalent.

- 9) $10010110_2 = \underline{\hspace{2cm}}_{(10)}$ 10) $10001011_2 = \underline{\hspace{2cm}}_{(10)}$
- 11) $1100111_2 = \underline{\hspace{2cm}}_{(10)}$ 12) $1100100_2 = \underline{\hspace{2cm}}_{(10)}$
- 13) $1000000_2 = \underline{\hspace{2cm}}_{(10)}$ 14) $11000101_2 = \underline{\hspace{2cm}}_{(10)}$
- 15) $11010010_2 = \underline{\hspace{2cm}}_{(10)}$ 16) $1010100_2 = \underline{\hspace{2cm}}_{(10)}$

Converting Decimal and Hexadecimal Numbers

Convert the given Decimal number to its Hexadecimal equivalent.

- 1) $82256_{(10)} = \underline{\hspace{2cm}}_{(16)}$ 2) $53977_{(10)} = \underline{\hspace{2cm}}_{(16)}$
- 3) $12815_{(10)} = \underline{\hspace{2cm}}_{(16)}$ 4) $27296_{(10)} = \underline{\hspace{2cm}}_{(16)}$
- 5) $47778_{(10)} = \underline{\hspace{2cm}}_{(16)}$ 6) $39501_{(10)} = \underline{\hspace{2cm}}_{(16)}$
- 7) $51484_{(10)} = \underline{\hspace{2cm}}_{(16)}$ 8) $59777_{(10)} = \underline{\hspace{2cm}}_{(16)}$

Convert the given Hexadecimal to its Decimal equivalent.

- 9) $2D7E_{(16)} = \underline{\hspace{2cm}}_{(10)}$ 10) $BE5F_{(16)} = \underline{\hspace{2cm}}_{(10)}$
- 11) $514E_{(16)} = \underline{\hspace{2cm}}_{(10)}$ 12) $535A_{(16)} = \underline{\hspace{2cm}}_{(10)}$
- 13) $EA52_{(16)} = \underline{\hspace{2cm}}_{(10)}$ 14) $BA5A_{(16)} = \underline{\hspace{2cm}}_{(10)}$
- 15) $B7E5_{(16)} = \underline{\hspace{2cm}}_{(10)}$ 16) $56B8_{(16)} = \underline{\hspace{2cm}}_{(10)}$

Variables

>

Binary calculations

Understand what data is in programming and how C stores it using variables.**Learn how to declare, initialize, and use variables in simple programs.**

Hexadecimal Addition and Subtraction

Add or Subtract the given Hexadecimal numbers.

1)
$$\begin{array}{r} \text{F40C} \\ - 9\text{BFB} \\ \hline \end{array}$$

2)
$$\begin{array}{r} \text{F40C} \\ - 749\text{C} \\ \hline \end{array}$$

3)
$$\begin{array}{r} 749\text{C} \\ + \text{BEA5} \\ \hline \end{array}$$

4)
$$\begin{array}{r} \text{E538} \\ - \text{BEA5} \\ \hline \end{array}$$

5)
$$\begin{array}{r} \text{E538} \\ - 589\text{C} \\ \hline \end{array}$$

6)
$$\begin{array}{r} 589\text{C} \\ + 9\text{AD4} \\ \hline \end{array}$$

7)
$$\begin{array}{r} \text{C394} \\ - 9\text{AD4} \\ \hline \end{array}$$

8)
$$\begin{array}{r} \text{C394} \\ - 4546 \\ \hline \end{array}$$

9)
$$\begin{array}{r} 4546 \\ + 5\text{D21} \\ \hline \end{array}$$

10)
$$\begin{array}{r} 5\text{D21} \\ - 3\text{C2D} \\ \hline \end{array}$$

11)
$$\begin{array}{r} 6429 \\ - 3\text{C2D} \\ \hline \end{array}$$

12)
$$\begin{array}{r} 6429 \\ - 2053 \\ \hline \end{array}$$

13)
$$\begin{array}{r} 2053 \\ + 5\text{DBE} \\ \hline \end{array}$$

14)
$$\begin{array}{r} \text{E193} \\ - 5\text{DBE} \\ \hline \end{array}$$

15)
$$\begin{array}{r} \text{EF88} \\ - \text{E193} \\ \hline \end{array}$$

16)
$$\begin{array}{r} \text{EF88} \\ - 88\text{DA} \\ \hline \end{array}$$

Bitwise Operations

1)
$$\begin{array}{r} 00101100 \\ \text{NOR } 00100100 \\ \hline \end{array}$$

2)
$$\begin{array}{r} 00100100 \\ \text{XOR } 00011001 \\ \hline \end{array}$$

3)
$$\begin{array}{r} 00011001 \\ \text{NXOR } 01110010 \\ \hline \end{array}$$

4)
$$\begin{array}{r} 01110010 \\ \text{NAND } 01010111 \\ \hline \end{array}$$

5)
$$\begin{array}{r} 01010111 \\ \text{XOR } 00011010 \\ \hline \end{array}$$

6)
$$\begin{array}{r} 00011010 \\ \text{AND } 00011111 \\ \hline \end{array}$$

7)
$$\begin{array}{r} 00011111 \\ \text{NXOR } 00110100 \\ \hline \end{array}$$

8)
$$\begin{array}{r} 00110100 \\ \text{NAND } 01101111 \\ \hline \end{array}$$

9)
$$\begin{array}{r} 01101111 \\ \text{AND } 00000010 \\ \hline \end{array}$$

10)
$$\begin{array}{r} 00000010 \\ \text{OR } 01111001 \\ \hline \end{array}$$

11)
$$\begin{array}{r} 01111001 \\ \text{AND } 01010000 \\ \hline \end{array}$$

12)
$$\begin{array}{r} 01010000 \\ \text{NAND } 00001111 \\ \hline \end{array}$$

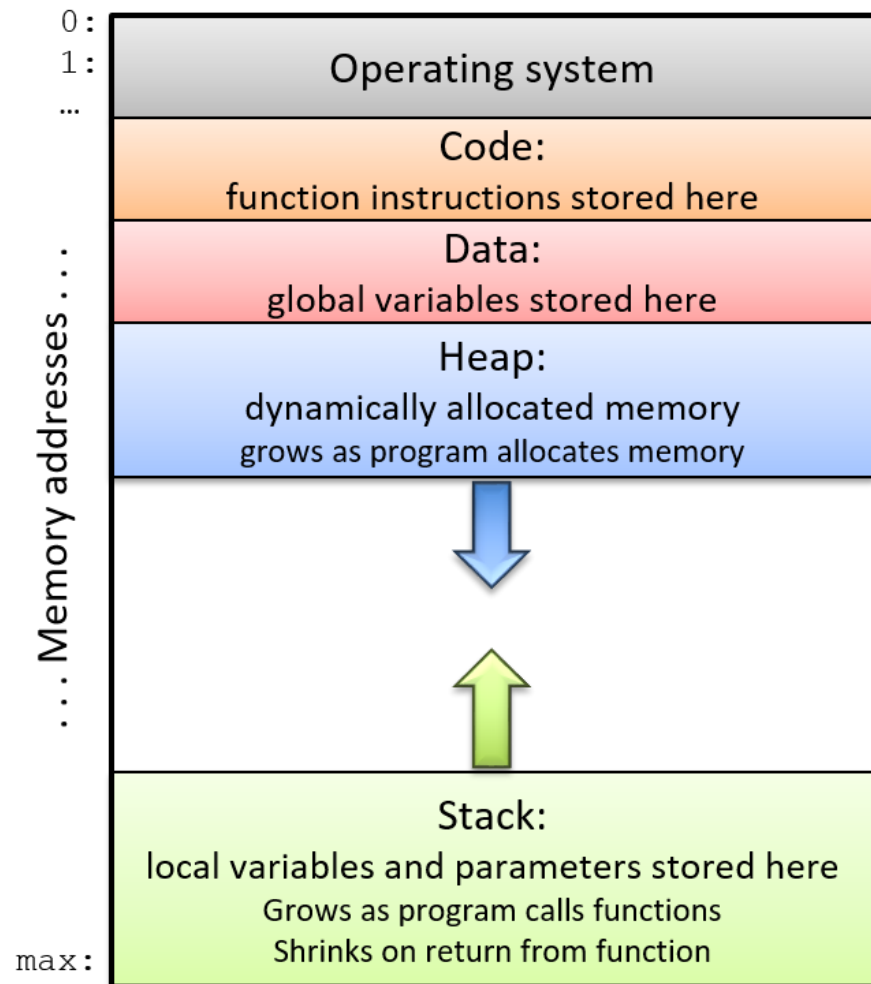
13)
$$\begin{array}{r} 00001111 \\ \text{OR } 01110001 \\ \hline \end{array}$$

14)
$$\begin{array}{r} 01110001 \\ \text{NOR } 01000011 \\ \hline \end{array}$$

15)
$$\begin{array}{r} 01000011 \\ \text{XOR } 00110101 \\ \hline \end{array}$$

16)
$$\begin{array}{r} 00110101 \\ \text{NOR } 01000110 \\ \hline \end{array}$$

Parts of Program Memory



In program:

(1) encounters variable definition`int a;`*(3) The compiler
remember the first
location of memory
space reserved for a**Symbol Table*

Name	Address	Size	Type
a	2400	4	int

What happens in the computer:

Memory

*(2) Find free space
and reserve it to
store value of a*2400
2401
2402
2403
2404

Scalar Types				
Type category	Type (C-syntax)	Format specifier ¹	Description ²	Data range ³
Integer	<u>s</u> hort	%hi	Small integer (≥ 4 bits)	[−32 767 ; +32 767]
	unsigned <u>s</u> hort	%hi		[0 ; 65 535]
	<u>i</u> nt	%i	Basic integer (≥ 16 bits)	[−32 767 ; +32 767]
	unsigned <u>i</u> nt	%u		[0 ; 65 535]
	<u>l</u> ong	%li	Long integer (≥32 bits)	[−2 147 483 647 ; +2 147 483 647]
	unsigned <u>l</u> ong	%lu		[0 ; 4 294 967 295]
	<u>long long</u>	%lli	Long integer (≥64 bits)	[−9 223 372 036 854 775 807 ; +9 223 372 036 854 775 807]
	unsigned <u>long long</u>	%llu		[0 ; 18 446 744 073 709 551 615]
Character (processed as integer)	<u>c</u> har	%c	Basic character. Processed as an integer behind the scenes (ASCII). Can be positive or negative. (≥ 4 bits)	[−127 ; +127] (signed) [0 ; 255] (unsigned)
	signed <u>c</u> har	%c		[−127 ; +127]
	unsigned <u>c</u> har	%c		[0 ; 255]
Real (decimal)	float	%f	(≥32 bits)	
	double	%lf	(≥64 bits)	
	long double	%lf	(≥128 bits)	
Boolean	bool	%d	A true/false variable (1 bit)	[false ; true]

Variables



Range & bit precision

Scalar Types			
Type category	Type (C-syntax)	Format specifier ¹	Description ²
Integer	<u>s</u> hort	%hi	Small integer (≥ 4 bits)
	unsigned <u>s</u> hort	%hi	
	<u>i</u> nt	% <u>i</u>	Basic integer (≥ 16 bits)
	unsigned <u>i</u> nt	%u	
	<u>l</u> ong	%li	Long integer (≥32 bits)
	unsigned <u>l</u> ong	% <u>l</u> <u>u</u>	
	<u>l</u> ong <u>l</u> ong	% <u>l</u> <u>l</u> <u>i</u>	Long integer (≥64 bits)
	unsigned <u>l</u> ong <u>l</u> ong	% <u>l</u> <u>l</u> <u>u</u>	
Character <i>(processed as integer)</i>	<u>c</u> har	%c	Basic character. Processed as an integer behind the scenes (ASCII). Can be positive or negative. (≥ 4 bits)
	signed <u>c</u> har	%c	
	unsigned <u>c</u> har	%c	
Real (decimal)	float	%f	(≥32 bits)
	double	% <u>l</u> <u>f</u>	(≥64 bits)
	long double	% <u>l</u> <u>f</u>	(≥128 bits)
Boolean	bool	%d	A true/false variable (1 bit)

Bit attribution

The number of bit depends on your OS architecture! (32 bits – 64 bits)

sizeof(int)

Use this function to know how much bits are attributed to each variables.

Example:

```
printf("Size of an 'int': %llu", sizeof(int));
```

Size of an 'int': 4

5 RULES!

1. Restricted to **letters**, **digits** and **underscore** (" _")
2. Must **NOT** start by a **digit**
3. "Case-sensitive": watch out for **capitalized** letters
4. Don't use **keywords** from C
5. Keep it **short** and concise (maximum 31 letters)

hello_world, helloWorld...

1dog, 2dog ...

(score != Score)

int, return ...

Constant variables

Write in **ALL_CAPS_WITH_UNDERSCORES**.

```
const float PI = 3.14159;  
#define MAX_SPEED 300
```

Variables

>

Naming convention in C (2/2)

TPE-004-1 | Introduction to Data and Variables

Understand what data is in programming and how C stores it using variables.

Learn how to declare, initialize, and use variables in simple programs.

Naming convention

camelCase

userName
totalPrice
fishDistributionResult

snake_case

user_name
total_price
fish_distribution_result

Name them CLEARLY

The variable name must be as indicative as possible, so that everyone know how and why to use your variables.

farenhS1 → **sensorFahrenheit_1**

Constants & Literals

>

"const"

TPE-004-2

Constants and Literals

Learn how to represent fixed values in programs and prevent changes to important data using constants.

Constants & Literals

>

"#define"

TPE-004-2 | Constants and Literals

Learn how to represent fixed values in programs and prevent changes to important data using constants.

Name	Operator	Arithmetic Operation	Syntax
Addition	+	Add two operands.	$x + y$
Subtraction	-	Subtract the second operand from the first operand.	$x - y$
Multiplication	*	Multiply two operands.	$x * y$
Division	/	Divide the first operand by the second operand.	x / y
<u>Modulus</u>	%	Calculate the remainder when the first operand is divided by the second operand.	$x \% y$

Tips

You can use “++” and “--” to update a value with +/- 1 easily.

Default

```
a = 3 ;  
b = a + 23 ;
```

Advanced

Assignment Operator	Shorthand Operation
$a = a + b$	$a += b$
$a = a - b$	$a -= b$
$a = a * b$	$a *= b$
$a = a / b$	$a /= b$
$a = a \% b$	$a \% = b$

Operator	Meaning
	Logical OR
&&	Logical AND
!	Logical NOT
	Bitwise logical OR
&	Bitwise logical AND
^	Bitwise logical exclusive OR

Operators and Expressions

>

Precedence and associativity

TPE-004-3 Operators and Expressions

Master the symbols that let you perform calculations and comparisons.

Understand how C evaluates expressions.

Operator	Name	Associativity
() [] . -> ++ --	Functional call Array element reference Indirect member selection Direct member selection Postfix increment Postfix decrement	Left to right
! ~ + - ++ -- & * sizeof (type)	Logical negation Bitwise 1's compliment Unary plus Unary minus Prefix increment Prefix decrement Address (of operand) Dereference Determine size in bytes Typecast	Right to Left
* / %	Multiply Divide Remainder	Left to right
+ -	Binary plus(Addition) Binary minus(subtraction)	Left to right
<< >>	Left shift Right shift	Left to right
< <= > >=	Less than Less than or equal Greater than Greater than or equal	Left to right

What is `printf()`

- `printf()` is a pre-defined function from the header `stdio.h`
- `Printf()` is used to write formatted output to console
- Example-

`printf("Hi There!");` output: `Hi There!`

`printf("My age is %d",23);` output: `My age is 23`

where `%d` is format specifier for integer

Scalar Types				
Type category	Type (C-syntax)	Format specifier ¹	Description ²	Data range ³
Integer	<code>short</code>	<code>%hi</code>	Small integer (≥ 4 bits)	<code>[-32 767 ; +32 767]</code>
	<code>unsigned short</code>	<code>%hi</code>		<code>[0 ; 65 535]</code>
	<code>int</code>	<code>%i</code>	Basic integer (≥ 16 bits)	<code>[-32 767 ; +32 767]</code>
	<code>unsigned int</code>	<code>%u</code>		<code>[0 ; 65 535]</code>
	<code>long</code>	<code>%li</code>	Long integer (≥32 bits)	<code>[-2 147 483 647 ; +2 147 483 647]</code>
	<code>unsigned long</code>	<code>%lu</code>		<code>[0 ; 4 294 967 295]</code>
	<code>long long</code>	<code>%lli</code>	Long integer (≥64 bits)	<code>[-9 223 372 036 854 775 807 ; +9 223 372 036 854 775 807]</code>
	<code>unsigned long long</code>	<code>%llu</code>		<code>[0 ; 18 446 744 073 709 551 615]</code>
Character (processed as integer)	<code>char</code>	<code>%c</code>	Basic character. Processed as an integer behind the scenes (ASCII). Can be positive or negative. (≥ 4 bits)	<code>[-127 ; +127] (signed)</code>
	<code>signed char</code>	<code>%c</code>		<code>[0 ; 255] (unsigned)</code>
	<code>unsigned char</code>	<code>%c</code>		<code>[-127 ; +127]</code>
Real (decimal)	<code>float</code>	<code>%f</code>	(≥32 bits)	
	<code>double</code>	<code>%lf</code>	(≥64 bits)	
	<code>long double</code>	<code>%lf</code>	(≥128 bits)	
Boolean	<code>bool</code>	<code>%d</code>	A true/false variable (1 bit)	<code>[false ; true]</code>

```
int age ;  
printf("Enter your age : ");  
scanf("%d", &age);
```



scanf reads an integer(a number)
which the user enters



scanf puts that read value
"At the address of" 'age' variable